

上海财经大学

信息管理与工程学院

拟牛顿法

优化理论与算法 第八章

王震

2026

本讲主线

上一讲介绍了牛顿法：

$$d^k = -[\nabla^2 f(x^k)]^{-1} \nabla f(x^k).$$

它利用真实 Hessian 曲率信息，局部收敛很快，但每步计算代价高。

本讲关注折中方案：

不直接计算 **Hessian**，而是用梯度变化逐步学习曲率。

曲率

用 s_k 与 y_k 描述位置和梯度的变化。

更新

通过割线条件构造 Hessian 近似。

实践

BFGS 与 L-BFGS 是最常用的拟牛顿方法。

本章目录

为什么需要拟牛顿法

割线条件

拟牛顿法基本框架

BFGS 方法

L-BFGS 与方法比较

目录

为什么需要拟牛顿法

割线条件

拟牛顿法基本框架

BFGS 方法

L-BFGS 与方法比较

牛顿法的代价

牛顿法每一步需要求解

$$\nabla^2 f(x^k) d^k = -\nabla f(x^k).$$

这带来三个主要成本。

计算

显式计算 Hessian 通常需要大量二阶导数。

存储

$n \times n$ Hessian 的存储代价约为 $\mathcal{O}(n^2)$ 。

求解

解稠密线性系统的代价约为 $\mathcal{O}(n^3)$ 。

当 n 很大时，真实 Hessian 往往太贵，但完全放弃曲率信息又会损失效率。

拟牛顿法的基本思想

拟牛顿法试图构造矩阵 B_k 或 H_k ，分别近似 Hessian 与逆 Hessian：

$$B_k \approx \nabla^2 f(x^k), \quad H_k \approx [\nabla^2 f(x^k)]^{-1}.$$

于是搜索方向可写为

$$d^k = -H_k \nabla f(x^k).$$

核心思想

不重新计算 Hessian，而是利用每一步的位移和梯度变化，对曲率近似矩阵做低成本更新。

- ▶ 比梯度下降多利用曲率信息；
- ▶ 比牛顿法少计算和存储真实 Hessian；
- ▶ 在许多光滑优化问题中有很好的实践表现。

拟牛顿法想“学习”什么

先看一维情形。若函数足够光滑，则二阶导数可以用导数变化近似：

$$f''(x) \approx \frac{f'(x^{k+1}) - f'(x^k)}{x^{k+1} - x^k}.$$

也就是说，只要知道两个点的梯度，就能估计这段路上的曲率。

多维情形中，变量和梯度都是向量。拟牛顿法做的事情可以理解为：

用“走了多远”和“梯度变了多少”来估计局部曲率。

课堂口径

牛顿法是“直接看 Hessian”；拟牛顿法是“通过连续几步的梯度变化，把 Hessian 慢慢学出来”。

目录

为什么需要拟牛顿法

割线条件

拟牛顿法基本框架

BFGS 方法

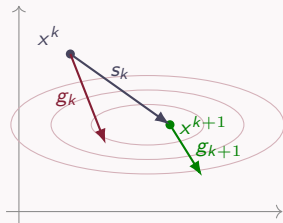
L-BFGS 与方法比较

位移与梯度差

从第 k 步到第 $k+1$ 步, 定义

$$s_k = x^{k+1} - x^k, \quad y_k = \nabla f(x^{k+1}) - \nabla f(x^k).$$

- ▶ s_k 记录变量移动的方向和长度;
- ▶ y_k 记录梯度如何变化;
- ▶ 梯度变化中包含了沿 s_k 方向的曲率信息。



一个二次函数例子： s_k, y_k 真的包含曲率

考虑二次函数

$$f(x) = \frac{1}{2}x^T Qx, \quad Q = \begin{bmatrix} 4 & 0 \\ 0 & 1 \end{bmatrix}.$$

它的梯度为

$$\nabla f(x) = Qx.$$

因此

$$y_k = \nabla f(x^{k+1}) - \nabla f(x^k) = Q(x^{k+1} - x^k) = Qs_k.$$

如果某一步

$$s_k = \begin{bmatrix} 1 \\ 2 \end{bmatrix},$$

那么

$$y_k = Qs_k = \begin{bmatrix} 4 \\ 2 \end{bmatrix}.$$

从 **Taylor** 展开到割线条件

由梯度的 Taylor 展开,

$$\nabla f(x^{k+1}) \approx \nabla f(x^k) + \nabla^2 f(x^{k+1})(x^{k+1} - x^k).$$

因此有近似关系

$$y_k = \nabla f(x^{k+1}) - \nabla f(x^k) \approx \nabla^2 f(x^{k+1})s_k.$$

定义 2.1: 割线条件

若 B_{k+1} 近似 Hessian, 则希望它满足

$$B_{k+1}s_k = y_k.$$

若 H_{k+1} 近似逆 Hessian, 则希望它满足

$$H_{k+1}y_k = s_k.$$

割线条件的直观含义

割线条件

$$B_{k+1}s_k = y_k$$

可以读作：

新矩阵 B_{k+1} 至少要在刚走过的方向上解释梯度变化。

如果用 B_{k+1}

B_{k+1} 近似 Hessian。

它把“位置变化” s_k 映射成“梯度变化” y_k 。

如果用 H_{k+1}

H_{k+1} 近似逆 Hessian。

它把“梯度变化” y_k 映射回“位置变化” s_k 。

所以 s_k, y_k 是拟牛顿法每一步最核心的数据。

割线条件还不够

割线条件只给出一个向量方程：

$$B_{k+1}s_k = y_k.$$

但 B_{k+1} 是一个矩阵，未知量很多，因此满足割线条件的矩阵并不唯一。

拟牛顿更新的设计目标

- ▶ 满足割线条件，保留最新曲率信息；
- ▶ 尽量少偏离旧矩阵 B_k 或 H_k ；
- ▶ 保持对称性；
- ▶ 在适当条件下保持正定性，保证下降方向。

不同的设计原则会导出不同的拟牛顿公式，最经典的是 DFP 与 BFGS。

为什么还要“尽量少改动”

割线条件只要求

$$B_{k+1}s_k = y_k.$$

但在二维中， B_{k+1} 有 4 个元素；若要求对称，也还有 3 个未知量。一个向量方程只给出 2 个约束，因此仍有很多选择。

拟牛顿更新通常还要求：

$$B_{k+1} \text{ 不要离 } B_k \text{ 太远.}$$

直觉

每一步只获得了一小段新的曲率信息，所以不要把之前积累的曲率近似完全推翻，而是在旧矩阵上做一个低秩修正。

BFGS 可以看作在“满足新曲率信息”和“保持旧矩阵稳定”之间取得平衡。

目录

为什么需要拟牛顿法

割线条件

拟牛顿法基本框架

BFGS 方法

L-BFGS 与方法比较

使用逆 Hessian 近似的迭代

若 $H_k \approx [\nabla^2 f(x^k)]^{-1}$, 则拟牛顿方向为

$$d^k = -H_k g_k, \quad g_k = \nabla f(x^k).$$

再结合线搜索:

$$x^{k+1} = x^k + \alpha_k d^k.$$

算法: 拟牛顿法框架

给定 x^0 与正定矩阵 H_0 。对 $k = 0, 1, 2, \dots$:

1. 计算 $g_k = \nabla f(x^k)$;
2. 令 $d^k = -H_k g_k$;
3. 线搜索得到步长 α_k ;
4. 更新 $x^{k+1} = x^k + \alpha_k d^k$;
5. 计算 $s_k = x^{k+1} - x^k$, $y_k = g_{k+1} - g_k$;

拟牛顿法如何从 $H_0 = I$ 开始

实际中常取

$$H_0 = I.$$

此时第一步方向为

$$d^0 = -H_0 g_0 = -g_0,$$

也就是普通负梯度方向。

从第二步开始，算法已经知道

$$s_0 = x^1 - x^0, \quad y_0 = g_1 - g_0.$$

于是可以调整 H_1 ，让它满足

$$H_1 y_0 = s_0.$$

直观过程

拟牛顿法一开始像 GD；随着迭代进行，它不断积累 (s_i, y_i) ，方向逐渐带有曲率校正。

为什么要求正定性

如果 $H_k \succ 0$, 则方向

$$d^k = -H_k g_k$$

满足

$$g_k^T d^k = -g_k^T H_k g_k < 0 \quad (g_k \neq 0).$$

因此 d^k 是下降方向。

推论 3.1: 正定性与下降方向

若 $H_k \succ 0$, 则拟牛顿方向 $d^k = -H_k g_k$ 是 f 在 x^k 处的下降方向。

所以拟牛顿更新不仅要满足割线条件, 还希望从 $H_k \succ 0$ 推出 $H_{k+1} \succ 0$ 。

目录

为什么需要拟牛顿法

割线条件

拟牛顿法基本框架

BFGS 方法

L-BFGS 与方法比较

BFGS 逆 Hessian 更新

BFGS 是最常用的拟牛顿方法之一。记

$$\rho_k = \frac{1}{y_k^T s_k}.$$

逆 Hessian 近似的 BFGS 更新为

$$H_{k+1} = (I - \rho_k s_k y_k^T) H_k (I - \rho_k y_k s_k^T) + \rho_k s_k s_k^T.$$

理解公式

这个更新只使用 H_k 、 s_k 、 y_k ，避免计算真实 Hessian，同时将最新曲率信息写入 H_{k+1} 。

BFGS 公式怎么读

BFGS 公式看起来复杂，但它主要完成三件事。

保留旧信息

更新从旧矩阵 H_k 出发，不重新计算 Hessian。

写入新曲率

利用 s_k, y_k 强制满足

$$H_{k+1}y_k = s_k.$$

保持稳定

在 $y_k^T s_k > 0$ 时保持 $H_{k+1} \succ 0$ ，从而给出下降方向。

所以课堂上不必把公式背下来，更重要的是理解它用 (s_k, y_k) 更新曲率近似。

BFGS 的正定性

BFGS 的重要性质是能够保持正定性。

定理 4.1: BFGS 正定保持

若 $H_k \succ 0$, 且

$$y_k^T s_k > 0,$$

则由 BFGS 更新得到的 H_{k+1} 仍满足

$$H_{k+1} \succ 0.$$

条件 $y_k^T s_k > 0$ 常称为曲率条件。对强凸函数并配合合适的 Wolfe 线搜索时, 该条件通常可以得到保证。

为什么重要

正定性保证 $d^k = -H_k g_k$ 是下降方向, 这是线搜索和全局收敛分析的重要基础。

曲率条件 $y_k^T s_k > 0$ 是什么

注意

$$y_k^T s_k = (g_{k+1} - g_k)^T (x^{k+1} - x^k).$$

它衡量的是：沿着刚走过的方向，梯度变化是否体现出“正曲率”。

对严格凸二次函数 $f(x) = \frac{1}{2}x^T Qx$ ，有

$$y_k = Qs_k,$$

因此

$$y_k^T s_k = s_k^T Qs_k > 0, \quad s_k \neq 0.$$

课堂口径

如果 $y_k^T s_k > 0$ ，说明刚走过的方向上函数像一个向上的碗；BFGS 就能把这种正曲率写入 H_{k+1} 。

BFGS 的实践特点

BFGS 位于梯度下降和牛顿法之间。

优点

- ▶ 不需要显式 Hessian;
- ▶ 方向质量通常明显好于 GD;
- ▶ 局部可达到超线性收敛;
- ▶ 中等规模问题上非常有效。

局限

- ▶ 仍需存储 $n \times n$ 矩阵;
- ▶ 每步矩阵向量乘法约 $\mathcal{O}(n^2)$;
- ▶ 对超大规模问题仍可能太贵。

这就引出了大规模优化中更常用的 L-BFGS。

目录

为什么需要拟牛顿法

割线条件

拟牛顿法基本框架

BFGS 方法

L-BFGS 与方法比较

L-BFGS: 有限记忆思想

BFGS 需要保存完整矩阵 H_k , 存储代价为 $\mathcal{O}(n^2)$ 。L-BFGS 的思想是:

不显式保存 H_k , 只保存最近 m 组 (s_i, y_i) 。

- ▶ m 通常远小于 n , 例如 $5 \sim 20$;
- ▶ 方向 $d^k = -H_k g_k$ 通过 two-loop recursion 间接计算;
- ▶ 存储代价从 $\mathcal{O}(n^2)$ 降为 $\mathcal{O}(mn)$;
- ▶ 非常适合大规模光滑无约束优化。

典型应用

逻辑回归、最大熵模型、光滑机器学习模型训练、大规模参数估计。

L-BFGS 的 two-loop recursion 在做什么

L-BFGS 不保存完整 H_k ，但仍要计算方向

$$d^k = -H_k g_k.$$

它用最近 m 组历史信息

$$(s_{k-m}, y_{k-m}), \dots, (s_{k-1}, y_{k-1})$$

间接完成这个乘法。

第一轮

从当前梯度 g_k 出发，按时间倒序“撤销”最近几次曲率影响。

第二轮

再按时间正序把这些曲率信息加回来，得到近似的 $H_k g_k$ 。

它的好处是：只用向量内积和向量加减，不需要存储或形成 $n \times n$ 矩阵。

BFGS 与 L-BFGS 的实际差别

方法	保存内容	适合规模
BFGS	完整矩阵 $H_k \in \mathbb{R}^{n \times n}$	中等规模，变量数不太大
L-BFGS	最近 m 组向量 (s_i, y_i)	大规模，变量数很大

例如当 $n = 10000$ 时，完整矩阵有 10^8 个元素，存储和计算都很重；若 $m = 10$ ，L-BFGS 只保存约 $20mn = 200000$ 个向量元素量级的信息。

实践建议

中小规模可先考虑 BFGS；大规模光滑问题通常优先考虑 L-BFGS。

GD、Newton、BFGS 与 L-BFGS

方法	使用信息	每步代价	适用场景
GD	梯度	低	大规模、精度要求一般
Newton	梯度 + Hessian	高	中小规模、高精度
BFGS	梯度 + 曲率近似	中高	中等规模光滑优化
L-BFGS	梯度 + 有限记忆曲率	中低	大规模光滑优化

选择原则

如果 Hessian 可承受，Newton 很强；如果 Hessian 太贵但仍想利用曲率，优先考虑 BFGS 或 L-BFGS。

本讲小结

- ▶ 拟牛顿法通过矩阵 B_k 或 H_k 近似 Hessian 或逆 Hessian。

- ▶ 位移

$$s_k = x^{k+1} - x^k$$

与梯度差

$$y_k = \nabla f(x^{k+1}) - \nabla f(x^k)$$

携带了沿迭代方向的曲率信息。

- ▶ 割线条件 $B_{k+1}s_k = y_k$ 或 $H_{k+1}y_k = s_k$ 是拟牛顿更新的核心约束。
- ▶ BFGS 在 $y_k^T s_k > 0$ 时保持正定性，常配合 Wolfe 线搜索。
- ▶ L-BFGS 只保存有限历史信息，是大规模光滑优化中的常用方法。

从二阶方法到后续内容

这一单元形成了从一阶到二阶的算法链条：

GD $\longrightarrow$ 动量与加速 $\longrightarrow$ Newton $\longrightarrow$ BFGS/L-BFGS.

后续课程将继续讨论另一类困难：

目标函数不光滑，或变量受到约束时，如何设计优化算法？

下一阶段

近端梯度法、投影梯度法与约束优化的一阶方法。